



北京航天丰益信息技术有限公司

编码规范 2019 年版

2019 年 4 月

一、 规范目的

本规范旨在用规范文件的形式，对部门人员进行有效的规范管理，让程序员尽快而彻底地理解新的代码，使得最终的软件工程具有良好的风格和统一的结构，且使代码可读性强、易维护。

培养程序员良好的编码规范，提高工作效率。

二、 适用范围

1、本规范是为撰写程序而设计，所以适用于一切有关程序撰写的工作事项。对于具体的每个项目，可能需要对之进行裁剪和补存。

2、适用人员：公司所有参与编程的软件开发人员以及其他相关人员。

三、 前端编码规范

1. 开发工具

WebStorm

2. 开发架构

1). PC 端

jQuery、Bootstrap

2). 移动端

方案 1: Vue、Vue-Router、Vuex、Axios、webpack

方案 2: jQuery、Bootstrap

3. HTML 规范

- 1). doctype 声明使用 html5
- 2). 统一页面编码格式为 utf-8 ， meta 标签 charset 设置为 utf-8;
- 3). 标签、标签属性全部小写。所有 html 标签必须有结束符，, <col />, <base />, <link />, <meta />, <input /> 除外。
- 4). 对于普通 class 或者 id 命名，统一使用小写字母，第一个字符必须为字母，连接符用中划线 _。
- 5). css 引用置于头部<head>标签内。
- 6). js 引用置于底部</body>标签前。
- 7). 开发时页面原则上不内嵌 style、script 代码，如特殊情况请标明并注释。
- 8). 缩进以 4 个空格为基本单位，为每个块级元素或表格元素标签新起一行，并且对每个子元素进行缩进。

4. CSS 代码规范

- 1). 样式类名全部用小写，首字符必须是字母，禁止数字或其他特殊字符。由以字母开头的小写字母（a-z）、数字（0-9）、中划线（-）组成。
- 2). 命名可以是单个单词，也可以是组合单词，要求能够描述清楚模块和元素的含义，使其具有语义化。避免使用 123456..., red, blue, left, right 之类的（如颜色、字号大小等）矢量命名，如 class="left-news"、class="2"，以避免当状态改变时名称失去意义。尽量用单个单词简单描述 class 名称。
- 3). 双单词或多单词组合方式：形容词-名词、命名空间-名次、命名空间-形容词-名词。例如：news-list、mod-feeds、mod-my-feeds、cell-title。
- 4). 样式使用竖排，不要使用横排以及 n 级缩进等。
- 5). class selector 层级尽量控制在 5 层以内。
- 6). 页面框架命名，一般具有唯一性，推荐用 ID 命名。如：头部（header）、主体（main）、脚部（footer）、容器（wrapper）、侧栏（sidebar）、栏目（column）、

布局 (layout)。

7). 模块结构命名。如：新闻模块 (mod-news)、标题栏 (title)、内容 (content)、次级内容 (sub-content)。

8). 导航结构命名。如：导航 (nav)、主导航 (main-nav)、子导航 (sub-nav)、顶部导航 (top-nav)、菜单 (menu)、子菜单 (sub-menu)。

5. JavaScript 代码规范

1). 所有 js, css 文件头部，必须有文件注释，用于简要说明：文件的主要功能 (含模块信息)、作者 (含作者邮箱)、版权等信息。

2). 代码需要保持适当的缩进。缩进请使用 Tab 缩进 (1 tab = 4 spaces, 可以在编辑器设置) 空格缩进显示以兼容不同 IDE, 不同操作平台的差异。

3). (建议) 每行代码不要超过 80 字符。当一条语句一行写不下时，折行。

4). 语句必须以分号作为结束符，不能忽略分号。

5). 变量使用 var、let 语句声明，变量使用前必须先声明。

6). 避免使用全局变量，如果不能避免则尽量减少数量。

7). 奇淫技巧必须注释，注释应该更多地体现为什么要这样，而不是会怎么样。

8). 返回的数据以标准的 JSON 格式来书写。

四、 后台编码规范

1. 开发工具

Spring tool suite (基于 eclipse)

2. 开放框架

基本框架：Spring Boot 2.1.0+

数据库框架：Spring Data

安全框架：Spring Security

3. 命名风格

- 1). **【强制】**方法名、参数名、成员变量、局部变量都统一使用 lowerCamelCase 风格，必须遵从 驼峰形式。

正例：localValue / getHttpMessage() / inputUserId。

- 2). **【强制】**常量命名全部大写，单词间用下划线隔开，力求语义表达完整清楚，不要嫌名字长。

正例：MAX_STOCK_COUNT

反例：MAX_COUNT

- 3). **【强制】**抽象类命名使用 Abstract 或 Base 开头；异常类命名使用 Exception 结尾；测试类命名以它要测试的类的名称开始，以 Test 结尾

- 4). **【强制】**中括号是数组类型的一部分，数组定义如下：String[] args;

- 5). **【强制】**POJO 类中布尔类型的变量，都不要加 is，否则部分框架解析会引起序列化错误

反例：定义为基本数据类型 Boolean isDeleted; 的属性，它的方法也是 isDeleted()，RPC 框架在反向解析的时候，“以为”对应的属性名称是 deleted，导致属性获取不到，进而抛出异常

- 6). **【推荐】**接口类中的方法和属性不要加任何修饰符号(public 也不要加)，保持代码的简洁性，并加上有效的 Javadoc 注释。尽量不要在接口里定义变量，如果一定要定义变量，肯定是与接口方法相关，并且是整个应用的基础常量。

正例：接口方法签名：void f();

接口基础常量表示：String COMPANY = "alibaba";

反例：接口方法定义：public abstract void f();

说明：JDK8 中接口允许有默认实现，那么这个 default 方法，是对所有实现类都有价值的默认实现。

- 7). **【参考】**枚举类名建议带上 Enum 后缀，枚举成员名称需要全大写，单词间用下划线隔开

8). 【参考】各层命名规约

A) Service/DAO 层方法命名规约

- 1) 获取单个对象的方法用 get 做前缀。
- 2) 获取多个对象的方法用 list 做前缀。
- 3) 获取统计值的方法用 count 做前缀。
- 4) 插入的方法用 save/insert 做前缀。
- 5) 删除的方法用 remove/delete 做前缀。
- 6) 修改的方法用 update 做前缀

4. 常量定义

- 1). 【推荐】不要使用一个常量类维护所有常量，按常量功能进行归类，分开维护。

说明：大而全的常量类，非得使用查找功能才能定位到修改的常量，不利于理解和维护

正例：缓存相关常量放在类 CacheConsts 下；系统配置相关常量放在类 ConfigConsts 下

- 2). 【推荐】如果变量值仅在一个范围内变化，且带有名称之外的延伸属性，定义为枚举类。下面 正例中的数字就是延伸信息，表示星期几。

正例：

```
public Enum { MONDAY(1), TUESDAY(2), WEDNESDAY(3), THURSDAY(4), FRIDAY(5), SATURDAY(6), SUNDAY(7);}
```

5. 代码格式

- 1). 【强制】大括号的使用约定。如果是大括号内为空，则简洁地写成 {} 即可，不需要换行；如果 是非空代码块则：

- 1) 左大括号前不换行。
- 2) 左大括号后换行。
- 3) 右大括号前换行。
- 4) 右大括号后还有 else 等代码则不换行；表示终止的右大括号后必须换行。

- 2). **【强制】** 左小括号和字符之间不出现空格；同样，右小括号和字符之间也不出现空格。详见 第 5 条下方正例提示
- 3). **【强制】** if/for/while/switch/do 等保留字与括号之间都必须加空格。
- 4). **【强制】** 任何二目、三目运算符的左右两边都需要加一个空格。
说明：运算符包括赋值运算符=、逻辑运算符&&、加减乘除符号等
- 5). **【强制】** 采用 4 个空格缩进，禁止使用 tab 字符
- 6). **【强制】** 注释的双斜线与注释内容之间有且仅有一个空格
- 7). **【强制】** 单行字符数限制不超过 120 个，超出需要换行，换行时遵循如下原则
第二行相对第一行缩进 4 个空格，从第三行开始，不再继续缩进，参考示例
运算符与下文一起换行
方法调用的点符号与下文一起换行
方法调用时，多个参数，需要换行时，在逗号后进行。
在括号前不要换行
- 8). **【强制】** 方法参数在定义和传入时，多个参数逗号后边必须加空格
- 9). **【强制】** IDE 的 text file encoding 设置为 UTF-8；IDE 中文件的换行符使用 Unix 格式，不要使用 Windows 格式
- 10). **【推荐】** 没有必要增加若干空格来使某一行的字符与上一行对应位置的字符对齐
- 11). **【推荐】** 方法体内的执行语句组、变量的定义语句组、不同的业务逻辑之间或者不同的语义 之间插入一个空行。相同业务逻辑和语义之间不需要插入空行

6. 集合处理

- 1). **【强制】** 关于 hashCode 和 equals 的处理，遵循如下规则：
 - 1) 只要重写 equals，就必须重写 hashCode。
 - 2) 因为 Set 存储的是不重复的对象，依据 hashCode 和 equals 进行

判断，所以 Set 存储的 对象必须重写这两个方法。

3) 如果自定义对象做为 Map 的键,那么必须重写 hashCode 和 equals。

2). **【强制】** ArrayList 的 subList 结果不可强转成 ArrayList, 否则会抛出 ClassCastException 异常, 即 java.util.RandomAccessSubList cannot be cast to java.util.ArrayList

3). **【强制】** 在 subList 场景中, 高度注意对原集合元素个数的修改, 会导致子列表的遍历、增加、删除均会产生 ConcurrentModificationException 异常。

4). **【强制】** 使用集合转数组的方法, 必须使用集合的 toArray(T[] array), 传入的是类型完全 一样的数组, 大小就是 list.size()。

5). **【强制】** 使用工具类 Arrays.asList() 把数组转换成集合时, 不能使用其修改集合相关的方法, 它的 add/remove/clear 方法会抛出 UnsupportedOperationException 异常。

说明: asList 的返回对象是一个 Arrays 内部类, 并没有实现集合的修改方法。Arrays.asList 体现的是适配器模式, 只是转换接口, 后台的数据仍是数组。

```
String[] str = new String[] { "you", "wu" };
```

```
List list = Arrays.asList(str);
```

第一种情况: list.add("yangguanbao"); 运行时异常。

第二种情况: str[0] = "gujin"; 那么 list.get(0) 也会随之修改

6). **【强制】** 泛型通配符 <? extends T> 来接收返回的数据, 此写法的泛型集合不能使用 add 方法, 而 <? super T> 不能使用 get 方法, 做为接口调用赋值时易出错。

7). **【强制】** 不要在 foreach 循环里进行元素的 remove/add 操作。remove 元素请使用 Iterator 方式, 如果并发操作, 需要对 Iterator 对象加锁

8). **【强制】** 在 JDK7 版本及以上, Comparator 要满足如下三个条件, 不然 Arrays.sort, Collections.sort 会报 IllegalArgumentException 异常

9). **【推荐】** 集合初始化时, 指定集合初始值大小。

说明：HashMap 使用 `HashMap(int initialCapacity)` 初始化，

正例： $\text{initialCapacity} = (\text{需要存储的元素个数} / \text{负载因子}) + 1$ 。注意负载因子（即 `loader factor`）默认为 0.75，如果暂时无法确定初始值大小，请设置为 16（即默认值）。

反例：HashMap 需要放置 1024 个元素，由于没有设置容量初始大小，随着元素不断增加，容量 7 次被迫扩大，`resize` 需要重建 hash 表，严重影响性能。

- 10). 【推荐】使用 `entrySet` 遍历 Map 类集合 KV,而不是 `keySet` 方式进行遍历。

说明：`keySet` 其实是遍历了 2 次，一次是转为 `Iterator` 对象，另一次是从 `hashMap` 中取出 `key` 所对应的 `value`。而 `entrySet` 只是遍历了一次就把 `key` 和 `value` 都放到了 `entry` 中，效率更高。如果是 JDK8，使用 `Map.forEach` 方法。

- 11). 【推荐】高度注意 Map 类集合 K/V 能不能存储 `null` 值的情况，如下表格：

集合类	Key	Value	Super	说明
Hashtable	不允许为 null	不允许为 null	Dictionary	线程安全
ConcurrentHashMap	不允许为 null	不允许为 null	AbstractMap	锁分段技术 (JDK8:CAS)
TreeMap	不允许为 null	允许为 null	AbstractMap	线程不安全
HashMap	允许为 null	允许为 null	AbstractMap	线程不安全

- 12). 【参考】合理利用好集合的有序性(`sort`)和稳定性(`order`)，避免集合的无序性(`unsort`)和 不稳定性(`unorder`)带来的负面影响。

说明：有序性是指遍历的结果是按某种比较规则依次排列的。稳定性指集合每次遍历的元素次序是一定的。如：`ArrayList` 是 `order/unsort`；`HashMap` 是 `unorder/unsort`；`TreeSet` 是 `order/sort`。

- 13). 【参考】利用 Set 元素唯一的特性，可以快速对一个集合进行去重操作，避免使用 List 的 `contains` 方法进行遍历、对比、去重操作。

7. 并发处理

- 1). **【强制】**获取单例对象需要保证线程安全，其中的方法也要保证线程安全。说明：资源驱动类、工具类、单例工厂类都需要注意。
- 2). **【强制】**创建线程或线程池时请指定有意义的线程名称，方便出错时回溯
- 3). **【强制】**线程资源必须通过线程池提供，不允许在应用中自行显式创建线程。

说明：使用线程池的好处是减少在创建和销毁线程上所花的时间以及系统资源的开销，解决资源不足的问题。如果不使用线程池，有可能造成系统创建大量同类线程而导致消耗完内存或者“过度切换”的问题。

- 4). **【强制】**线程池不允许使用 `Executors` 去创建，而是通过 `ThreadPoolExecutor` 的方式，这样的处理方式让写的同学更加明确线程池的运行规则，规避资源耗尽的风险
- 5). **【强制】**`SimpleDateFormat` 是线程不安全的类，一般不要定义为 `static` 变量，如果定义为 `static`，必须加锁，或者使用 `DateUtils` 工具类。

正例：注意线程安全，使用 `DateUtils`。亦推荐如下处理：

```
private static final ThreadLocal<DateFormat> df = new ThreadLocal<DateFormat>() {  
    @Override  
    protected DateFormat initialValue() {  
        return new SimpleDateFormat("yyyy-MM-dd");  
    }  
};
```

说明：如果是 `JDK8` 的应用，可以使用 `Instant` 代替 `Date`，`LocalDateTime` 代替 `Calendar`，`DateTimeFormatter` 代替 `SimpleDateFormat`，官方给出的解释：`simple beautiful strong immutable thread-safe`。

- 6). **【强制】**高并发时，同步调用应该去考量锁的性能损耗。能用无锁数据结构，就不要用锁；能锁区块，就不要锁整个方法体；能用对象锁，就不要用类锁。

说明：尽可能使加锁的代码块工作量尽可能的小，避免在锁代码块中调用 `RPC` 方法

- 7). **【强制】**对多个资源、数据库表、对象同时加锁时，需要保持一致的加锁顺序，否则可能会造成死锁。

说明：线程一需要对表 A、B、C 依次全部加锁后才可以进行更新操作，那么线程二的加锁顺序 也必须是 A、B、C，否则可能出现死锁

- 8). **【强制】**并发修改同一记录时，避免更新丢失，需要加锁。要么在应用层加锁，要么在缓存加 锁，要么在数据库层使用乐观锁，使用 version 作为更新依据。

说明：如果每次访问冲突概率小于 20%，推荐使用乐观锁，否则使用悲观锁。乐观锁的重试次 数不得小于 3 次。

- 9). **【强制】**多线程并行处理定时任务时，Timer 运行多个 TimeTask 时，只要其中之一没有捕获 抛出的异常，其它任务便会自动终止运行，使用 ScheduledExecutorService 则没有这个问题

- 10). **【推荐】**使用 CountdownLatch 进行异步转同步操作，每个线程退出前必须调用 countDown 方法，线程执行代码注意 catch 异常，确保 countDown 方法被执行到，避免主线程无法执行 至 await 方法，直到超时才返回结果。

说明：注意，子线程抛出异常堆栈，不能在主线程 try-catch 到

- 11). **【推荐】**避免 Random 实例被多线程使用，虽然共享该实例是线程安全的，但会因竞争同一 seed 导致的性能下降。

说明：Random 实例包括 java.util.Random 的实例或者 Math.random() 的方式。正例：在 JDK7 之后，可以直接使用 API ThreadLocalRandom，而在 JDK7 之前，需要编码保 证每个线程持有一个实例。

- 12). **【推荐】**在并发场景下，通过双重检查锁（double-checked locking）实现延迟初始化的优 化问题隐患(可参考 The “Double-Checked Locking is Broken” Declaration)，推荐解 决方案中较为简单一种(适用于 JDK5 及以上版本)，将目标属性声明为 volatile 型

- 13). **【参考】**volatile 解决多线程内存不可见问题。对于一写多读，是可以解决变量同步问题， 但是如果多写，同样无法解决线程安全问题。如果是 count++ 操作，使用如下类实现： AtomicInteger count = new AtomicInteger(); count.addAndGet(1); 如果是 JDK8，推 荐使用 LongAdder 对象，比 AtomicLong 性能更好（减少乐观锁的重试次数）。

- 14). **【参考】**HashMap 在容量不够进行 resize 时由于高并发可能出现死链，导

致 CPU 飙升，在开发过程中可以使用其它数据结构或加锁来规避此风险。

- 15). **【参考】**ThreadLocal 无法解决共享对象的更新问题，ThreadLocal 对象建议使用 static 修饰。这个变量是针对一个线程内所有操作共享的，所以设置为静态变量，所有此类实例共享此静态变量，也就是说在类第一次被使用时装载，只分配一块存储空间，所有此类的对象(只要是这个线程内定义的)都可以操控这个变量。

8. 控制语句

- 1). **【强制】**在一个 switch 块内，每个 case 要么通过 break/return 等来终止，要么注释说明程序将继续执行到哪一个 case 为止；在一个 switch 块内，都必须包含一个 default 语句并且放在最后，即使它什么代码也没有。
- 2). **【强制】**在 if/else/for/while/do 语句中必须使用大括号。即使只有一行代码，避免采用单行的编码方式：if (condition) statements;
- 3). **【推荐】**表达异常的分支时，少用 if-else 方式，这种方式可以改写成：

```
if (condition) {  
    ...  
    return obj;  
}
```

// 接着写 else 的业务逻辑代码；

说明：如果非得使用 if()...else if()...else... 方式表达逻辑，**【强制】**避免后续代码维护困难，请勿超过 3 层。

正例：超过 3 层的 if-else 的逻辑判断代码可以使用卫语句、策略模式、状态模式等来实现，1

- 4). **【推荐】**除常用方法（如 getXxx/isXxx）等外，不要在条件判断中执行其它复杂的语句，将复杂逻辑判断的结果赋值给一个有意义的布尔变量名，以提高可读性。

说明：很多 if 语句内的逻辑相当复杂，阅读者需要分析条件表达式的最

终结果，才能明确什么样的条件执行什么样的语句，那么，如果阅读者分析逻辑表达式错误呢？

- 5). **【推荐】**循环体中的语句要考量性能，以下操作尽量移至循环体外处理，如定义对象、变量、获取数据库连接，进行不必要的 try-catch 操作（这个 try-catch 是否可以移至循环体外）
- 6). **【推荐】**接口入参保护，这种场景常见的是用于做批量操作的接口。
- 7). **【参考】**下列情形，需要进行参数校验：
 - 1) 调用频次低的方法。
 - 2) 执行时间开销很大的方法。此情形中，参数校验时间几乎可以忽略不计，但如果因为参数错误导致中间执行回退，或者错误，那得不偿失。
 - 3) 需要极高稳定性和可用性的方法。
 - 4) 对外提供的开放接口，不管是 RPC/API/HTTP 接口。
 - 5) 敏感权限入口。
- 8). **【参考】**下列情形，不需要进行参数校验：
 - 1) 极有可能被循环调用的方法。但在方法说明里必须注明外部参数检查要求。
 - 2) 底层调用频度比较高的方法。毕竟是像纯净水过滤的最后一道，参数错误不太可能到底层才会暴露问题。一般 DAO 层与 Service 层都在同一个应用中，部署在同一台服务器中，所以 DAO 的参数校验，可以省略。
 - 3) 被声明成 private 只会被自己代码所调用的方法，如果能够确定调用方法的代码传入参数已经做过检查或者肯定不会有问题，此时可以不校验参数。

9. 其它

- 1). **【强制】**在使用正则表达式时，利用好其预编译功能，可以有效加快正则匹配速度。说明：不要在方法体内定义：`Pattern pattern = Pattern.compile(规则);`
- 2). **【强制】**velocity 调用 POJO 类的属性时，建议直接使用属性名取值即可，

模板引擎会自动按 规范调用 POJO 的 `getXxx()`，如果是 `boolean` 基本数据类型变量（`boolean` 命名不需要加 `is` 前缀），会自动调用 `isXxx()` 方法。

说明：注意如果是 `Boolean` 包装类对象，优先调用 `getXxx()` 的方法。

- 3). **【强制】** 后台输送给页面的变量必须加 `#{var}`——中间的感叹号。

说明：如果 `var=null` 或者不存在，那么 `#{var}` 会直接显示在页面上。

- 4). **【强制】** 注意 `Math.random()` 这个方法返回是 `double` 类型，注意取值的范围 $0 \leq x < 1$ （能够 取到零值，注意除零异常），如果想获取整数类型的随机数，不要将 `x` 放大 10 的若干倍然后 取整，直接使用 `Random` 对象的 `nextInt` 或者 `nextLong` 方法。

- 5). **【强制】** 获取当前毫秒数 `System.currentTimeMillis()`；而不是 `new Date().getTime()`；说明：如果想获取更加精确的纳秒级时间值，使用 `System.nanoTime()` 的方式。在 `JDK8` 中，针对统计时间等场景，推荐使用 `Instant` 类。

- 6). **【推荐】** 不要在视图模板中加入任何复杂的逻辑。说明：根据 `MVC` 理论，视图的职责是展示，不要抢模型和控制器的活。

- 7). **【推荐】** 任何数据结构的构造或初始化，都应指定大小，避免数据结构无限增长吃光内存。

- 8). **【推荐】** 及时清理不再使用的代码段或配置信息。说明：对于垃圾代码或过时配置，坚决清理干净，避免程序过度臃肿，代码冗余。正例：对于暂时被注释掉，后续可能恢复使用的代码片断，在注释代码上方，统一规定使用三个斜杠（`///`）来说明注释掉代码的理由。

10. 异常处理

- 1). **【强制】** `Java` 类库中定义的一类 `RuntimeException` 可以通过预先检查进行规避，而不应该 通过 `catch` 来处理，

比如：`IndexOutOfBoundsException`，`NullPointerException` 等等。说明：

无法通过预检查的异常除外，如在解析一个外部传来的字符串形式数

字时，通过 `catch NumberFormatException` 来实现。

正例：`if (obj != null) {...}`

反例：`try { obj.method() } catch (NullPointerException e) {...}`

2). **【强制】**异常不要用来做流程控制，条件控制，因为异常的处理效率比条件分支低。

3). **【强制】**对大段代码进行 `try-catch`，这是不负责任的表现。`catch` 时请分清稳定代码和非稳定代码，稳定代码指的是无论如何不会出错的代码。对于非稳定代码的 `catch` 尽可能进行区分异常类型，再做对应的异常处理。

4). **【强制】**捕获异常是为了处理它，不要捕获了却什么都不处理而抛弃之，如果不想处理它，请将该异常抛给它的调用者。最外层的业务使用者，必须处理异常，将其转化为用户可以理解的内容。

5). **【强制】**有 `try` 块放到了事务代码中，`catch` 异常后，如果需要回滚事务，一定要注意手动回滚事务。

6). **【强制】**`finally` 块必须对资源对象、流对象进行关闭，有异常也要做 `try-catch`。

说明：如果 JDK7 及以上，可以使用 `try-with-resources` 方式。

7). **【强制】**不能在 `finally` 块中使用 `return`，`finally` 块中的 `return` 返回后方法结束执行，不会再执行 `try` 块中的 `return` 语句。

8). **【强制】**捕获异常与抛异常，必须是完全匹配，或者捕获异常是抛异常的父类。

说明：如果预期对方抛的是绣球，实际接到的是铅球，就会产生意外情况。

9). **【推荐】**方法的返回值可以为 `null`，不强制返回空集合，或者空对象等，必须添加注释充分说明什么情况下会返回 `null` 值。调用方需要进行 `null` 判断防止 NPE 问题。

说明：本手册明确防止 NPE 是调用者的责任。即使被调用方法返回空集合或者空对象，对调用者来说，也并非高枕无忧，必须考虑到远程调用失败、序列化失败、运行时异常等场景返回 `null` 的情况。

10). **【推荐】**防止 NPE，是程序员的基本修养，注意 NPE 产生的场景：

1) 返回类型为基本数据类型，`return` 包装数据类型的对象时，自动拆箱

有可能产生 NPE。

反例：public int f() { return Integer 对象}, 如果为 null, 自动解箱抛 NPE。

2) 数据库的查询结果可能为 null。

3) 集合里的元素即使 isEmpty(), 取出的数据元素也可能为 null。

4) 远程调用返回对象时, 一律要求进行空指针判断, 防止 NPE。

5) 对于 Session 中获取的数据, 建议 NPE 检查, 避免空指针。

6) 级联调用 obj.getA().getB().getC(); 一连串调用, 易产生 NPE。正

例: 使用 JDK8 的 Optional 类来防止 NPE 问题。

- 11). 【推荐】定义时区分 unchecked / checked 异常, 避免直接抛出 new RuntimeException(), 更不允许抛出 Exception 或者 Throwable, 应使用有业务含义的自定义异常。推荐业界已定义过的自定义异常, 如: DAOException / ServiceException 等。

- 12). 【参考】在代码中使用“抛异常”还是“返回错误码”, 对于公司外的 http/api 开放接口必须使用“错误码”; 而应用内部推荐异常抛出; 跨应用间 RPC 调用优先考虑使用 Result 方式, 封装 isSuccess() 方法、“错误码”、“错误简短信息”。

说明: 关于 RPC 方法返回方式使用 Result 方式的理由:

- 1) 使用抛异常返回方式, 调用方如果没有捕获到就会产生运行时错误。
2) 如果不加栈信息, 只是 new 自定义异常, 加入自己的理解的 error message, 对于调用端解决问题的帮助不会太多。如果加了栈信息, 在频繁调用出错的情况下, 数据序列化和传输的性能损耗也是问题。

- 13). 【参考】避免出现重复的代码 (Don't Repeat Yourself), 即 DRY 原则。说明: 随意复制和粘贴代码, 必然会导致代码的重复, 在以后需要修改时, 需要修改所有的副本, 容易遗漏。必要时抽取共性方法, 或者抽象公共类, 甚至是组件化。

正例: 一个类中有多个 public 方法, 都需要进行数行相同的参数校验操作, 这个时候请抽取: private boolean checkParam(DTO dto) {...}

五、 数据库编码规范

1. 库表规范

- 1). 库名、表名、字段名等，需使用与业务内容相关的英文词汇，需见名知意，多个单词组成的，中间以下划线分割
- 2). 表达是与否概念的字段，必须使用 `is_xxx` 的方式命名，数据类型是 `int(1)`，1 表示‘是’，0 表示‘否’。表达逻辑删除的字段名为 `is_deleted`，1 表示‘删除’，0 表示‘未删除’
- 3). 临时库、临时表名称，需要以 `tmp_` 日期为后缀。
例：`project_info_tmp20190223`
- 4). 避免使用 oracle 的关键字来命名。例 `desc`、`range`、`match` 等。
- 5). 命名只能使用英文字母、数字和下划线，并且不能以数字开头，字母全部小写，长度尽量不要超过 32 个字符。
- 6). 所有表、字段都要添加注释，标记型字段需注明主要值含义。例：`status` 字段，‘1-正常，0-非正常’
- 7). 所有表都要指定主键，并且主键索引名以 `pk_` 为前缀；唯一索引以 `uk_` 为前缀；外键约束以 `fk_` 为前缀；普通索引以 `idx_` 为前缀。
- 8). 表必备三个字段：`id`（主键）、`createdate`（创建时间）、`updatedate`（修改时间）
- 9). 字段需要给予适当的数据类型及存储长度，以便能节约数据库表空间、提高检索速度。例：姓名字段，设成 `varchar2(11)` 即可，存储长度不要设置过长
- 10). 同意意义的字段在不同表中定义应该相同。例 `report_id` 字段，在不同表中的类型、字段长度要相同

2. 索引规范

- 1). 业务上具有唯一特性的字段，即使是多个字段的组合，也必须建成唯一索引。
说明：不要以为唯一索引影响了 `insert` 速度，这个速度损耗可以忽略，

但提高查找速度是明显的

- 2). 超过三个表关联, 需要 join 的字段数据类型必须一致; 多表关联查询时, 保证被关联的字段需要有索引。说明: 即使双表 join 也要注意表索引、SQL 性能
- 3). 建组合索引的时候, 区分度最高的在最左边。正例: 如果 where a=? and b=? , a 列的几乎接近于唯一值, 那么只需要单建 idx_a 索引即可。说明: 存在非等号和等号混合判断条件时, 在建索引时, 请把等号条件的列前置。如: where a>? and b=? 那么即使 a 的区分度更高, 也必须把 b 放在索引的最前列
- 4). 防止因字段类型不同造成的隐式转换, 导致索引失效。

3. SQL 语句规范

- 1). 不要使用 count(列名)或 count(常量)来替代 count(*), count(*)是 SQL92 定义的 标准统计行数的语法, 跟数据库无关, 跟 NULL 和非 NULL 无关。说明: count(*)会统计值为 NULL 的行, 而 count(列名)不会统计此列为 NULL 值的行。
- 2). count(distinct col) 计算该列除 NULL 之外的不重复行数, 注意 count(distinct col1, col2) 如果其中一列全为 NULL, 那么即使另一列有不同的值, 也返回为 0。
- 3). 使用 NVL()来判断是否为 NULL 值。说明: NULL 与任何值的直接比较都为 NULL。 1) NULL<>NULL 的返回结果是 NULL, 而不是 false。 2) NULL=NULL 的返回结果是 NULL, 而不是 true。 3) NULL<>1 的返回结果是 NULL, 而不是 true。
- 4). 在代码中写分页查询逻辑时, 若 count 为 0 应直接返回, 避免执行后面的分页语句
- 5). 数据订正时, 删除和修改记录时, 要先 select, 避免出现误删除, 确认无误才能执 行更新语句
- 6). in 操作能避免则避免, 若实在避免不了, 需要仔细评估 in 后边的集合元素

数量，控制在 1000 个之内

7). 如果有全球化需要，所有的字符存储与表示，均以 utf-8 编码

8). TRUNCATE TABLE 比 DELETE 速度快，且使用的系统和事务日志资源少，但 TRUNCATE 无事务且不触发 trigger，有可能造成事故，故不建议在开发代码中使用此语句。说明：TRUNCATE TABLE 在功能上与不带 WHERE 子句的 DELETE 语句相同

六、 附则

公司程序员及相关人员须认真阅读并遵守《编码规范》的相关规定。

本规范服从国家有关法律法规。

本规范属于公司保密性文件，无论员工在公司工作或离职以后，均不得以任何形式向外传播、翻译、提供他人。

本规范的解释权、修改权归公司技术研发部所有。

北京航天丰益信息技术有限公司

2019 年 4 月 11 日